

Modular DBMS Engine -- Project Report

CMPE 321 -- Spring 2026 -- Project 3 -- Group 7

Submission Links (spec sec. 16)

Explanation video (Layer-by-layer walkthrough, spec section 11):
TODO_VIDEO_LINK_EXPLANATION

Analysis video (Comparative experiments, spec section 12): TODO_VIDEO_LINK_ANALYSIS

Source code (private GitHub repository, spec section 16): TODO_REPO_LINK

1. Overview

We built a modular DBMS engine composed of four independent layers: a Disk Space Manager (Layer 1), a Buffer Manager (Layer 2), a File & Index Manager (Layer 3), and a Query Processor (Layer 4). Each layer lives in its own Python package and exports a single class through its `__init__.py`. Layers communicate only via Result objects -- small dataclasses that carry both data and operation metadata. Because the contract between layers is a typed Result, the internal implementation of any layer can change without touching the others.

The engine is reconfigured by editing a single JSON file. Swapping LRU for MRU, hash_index for B+ tree, or doubling the buffer pool requires no code change. We exploit this in Section 6 to run three experiments comparing the available options.

Result objects (the inter-layer contract)

Every cross-layer call returns one of six dataclasses defined in `results.py`: `PageResult` (raw page bytes plus `io_performed` -- True on real disk I/O, False on cache hit); `WriteResult` (success flag, `old_data`, `new_data`; fed to the `log_write` stub on every write); `AllocResult` (id of a newly allocated logical page); `BufferResult` (wraps a `PageResult` and adds `cache_hit`, `evicted_page_id`, `dirty_writeback`); `RecordResult` (list of matching records plus `pages_accessed` and `index_nodes_visited`); `OpResult` (success/failure of write-style operations -- create, insert, delete -- with the same metadata fields).

Each Result also carries a status string (SUCCESS / FAILURE / ERROR) and an optional error message. Failures never propagate as exceptions -- every layer reports problems to the layer above through its Result, which is what lets the Query Processor log failure in `log.csv` without the engine crashing.

2. Layer 1 -- Disk Space Manager

The lowest layer. The only component that performs real file I/O. Knows nothing about records, indexes, or queries -- only raw fixed-size pages. Operates on per-relation binary files using `seek()` + single-page reads/writes; we never load an entire file into memory.

Per-file layout: every DSM-managed file starts with one DSM-owned header page (physical page 0), followed by content pages. Upper layers see logical pages 0, 1, 2, ...; internally we translate $\text{physical} = \text{logical} + 1$. This makes free-space bookkeeping a DSM secret -- upper layers never see or touch the header.

The header (physical page 0) stores a 4-byte magic constant DSM1, the total number of content pages, and a free list of logical page ids that have been deallocated and can be reused. With `page_size = 4096` the free list can hold 1021 entries in a single page -- plenty for our workloads. We chose a free list over a global bitmap because whole-page deallocations are rare; a short list is cheaper to maintain and store than a file-wide bitmap.

I/O counting and the `log_write` stub: two private methods, `_read_physical_page` and `_write_physical_page`, are the only places the counters change. Every header read counts too (it's real disk traffic). Spec section 4.1 requires a `log_write` stub: a callable invoked on every write. We keep it as an instance attribute (default no-op) so it can be replaced from outside for tests or for the Query Processor's logging needs.

3. Layer 2 -- Buffer Manager

In-memory page cache between Layer 3 and Layer 1. Implements both LRU and MRU replacement policies, tracks hit/miss/eviction/dirty-writeback counts, and exposes a strict pin/unpin protocol so upper layers can modify pages safely.

Replacement policies on one data structure: the pool is an `OrderedDict` keyed by `(file_id, page_id)`. This gives us $O(1)$ membership, $O(1)$ move-to-end, and $O(1)$ pop from either end. Convention: end (right side) = most-recently-used; front (left side) = least-recently-used. Every cache access moves its frame to the end. LRU evicts from the front (oldest); MRU evicts from the end (newest). One data structure, two policies -- only the eviction call differs.

Pin / unpin and the dirty bit: every `get_page` increments the frame's pin count. The caller must eventually call `unpin(file_id, page_id, dirty=...)`. The eviction picker skips pinned frames; when none is unpinned, that's a bug (a caller forgot to unpin) and we surface it loudly rather than silently corrupt state. Dirty frames are written back through the DSM on eviction or on `flush()`.

What the cache hit returns: a `BufferResult` wrapping a `PageResult` with `io_performed=False`, plus `cache_hit=True`, plus eviction info (`evicted_page_id`, `dirty_writeback`). The caller never has to ask the DSM whether I/O happened -- the Result tells them.

4. Layer 3 -- File & Index Manager

Understands records, relations (types), pages, and indexes. Always goes through the Buffer Manager for page access -- never calls `DiskSpaceManager` directly for data. Implements three interchangeable index strategies; the active one is chosen by `config["index_strategy"]`.

Schema and record serialization

A Schema dataclass holds the type's name, ordered field list ((name, "int"|"str") tuples), and the 0-indexed primary-key position. Per DESIGN.md, we chose: int = 4 bytes signed little-endian (<i); str = 32 bytes ASCII, null-padded, trimmed on read.

Worked example for the spec's house type (3 str + 3 int fields): record size = 3 x 32 + 3 x 4 = 108 bytes. With page_size = 4096 and the 16-byte page header, 37 records would fit geometrically -- but we cap at max_records_per_page = 10 (the spec's hard cap), so 10 slots actually used.

Slotted page layout

Each data page has a 16-byte header followed by fixed-offset slots:

- bytes 0..3 : page_id (sanity)
- bytes 4..7 : record_count
- bytes 8..9 : slot_bitmap (uint16; bit i = slot i occupied)
- bytes 10..15 : reserved / padding
- bytes 16.. : slot i at offset 16 + i * record_size

This is the unpacked slotted format: slot positions are fixed, and the bitmap tells us which slots are live. Inserts and deletes are O(1) on slot lookup and never shift records. The 10-bit bitmap fits in a uint16, sized for the 10-slot cap.

System catalog

Type metadata persists in catalog.dat, a normal DSM-managed paged file accessed through the Buffer Manager. Logical page 0 of the catalog stores a 4-byte type count followed by each Schema packed back-to-back via pack_schema. On engine restart, Catalog.bootstrap() reads this page and rebuilds the in-memory dict -- no other state has to be reconstructed.

Index strategies

heap_scan (baseline). No index. Inserts append to the first page with a free slot; lookups and range queries scan all pages. The index_nodes_visited counter stays at 0.

hash_index. Static hash on the primary key with 64 buckets. Each <type>.hidx file holds: logical page 0 = directory (64 x int32 = the first-page id of each bucket), logical pages 1..N = bucket pages with overflow chaining. Bucket pages store back-to-back (pk, data_page_id, slot_id) entries -- no bitmap; deletes compact by moving the last entry into the freed slot. Range queries fall back to heap_scan per spec section 7.2.

bplus_tree. B+ tree on the primary key with two node kinds in the same file. Logical page 0 of <type>.bidx is the tree header (root_page_id, height). Internal nodes store interleaved c0, k0, c1, k1, ..., k(n-1), c(n); leaves store back-to-back entries plus a next_leaf pointer for range scans. Splits propagate up; if the root splits, the tree grows a level. We cap per-node entries at max_records_per_page (default 10) so the tree actually has multiple levels in our workloads -- otherwise the natural fanout would keep everything in one root.

Index correctness across strategy switches. On engine startup the File & Index Manager always rebuilds the active index from the data file (deleting any pre-existing index file first). This guarantees correct behaviour when a previous run mutated the data under a different strategy and left the other strategy's index stale.

5. Layer 4 -- Query Processor

Top layer. Parses input commands, dispatches to the File & Index Manager, collects per-command statistics, writes results to output.txt, and logs every operation to log.csv.

Output files: output.txt is truncated at the start of each run and written on successful searches and on every explain. log.csv is opened in append mode and survives restarts as the spec requires. stats_output.txt is overwritten on every stats command (it's a snapshot, not a log).

Explain handler steps: (1) snapshot the layer counters, (2) compute a plan (strategy + estimated I/O via simple per-strategy heuristics), (3) write the PLAN block, (4) run the wrapped command and collect result lines, (5) write the RESULT block, (6) subtract the snapshot from the current counters and write the STATS block.

Failure handling: every process(line) call is wrapped in a top-level try/except. Bad commands (unknown verbs, wrong arity, duplicate primary keys, missing types, range on a string field, ...) produce a Result with success=False, are logged as failure, and produce no output. The engine never crashes.

6. Experiments

All experiments use a 6-field relation produced by workload_generator.py (PK + 2 strings + 3 ints, meeting the spec's 'at least 4 fields, 2+ ints' requirement). Workloads are deterministic -- fixed RNG seed 42 -- so every run is byte-identical. The full reproduction recipe is in record.txt.

Every number below is the total I/O (disk reads + writes) recorded in stats_output.txt after the workload, with a final stats command appended by the runner.

Experiment 1 -- LRU vs MRU

Setup: 200 records loaded, then 50 queries. Buffer pool = 8 frames. Strategy = heap_scan (every query is a full scan).

Workload	LRU I/Os	LRU Hit Rate	MRU I/Os	MRU Hit Rate
sequential	5105	17.4%	3245	51.4%
random	4697	19.8%	3023	52.5%

When a scan touches more pages than the pool can hold, LRU is the worst possible policy: it evicts exactly the page the next pass of the scan is about to need. MRU keeps the older pages and re-uses them on each pass. Our measured result is the textbook one: MRU produces ~37% fewer I/Os and almost 3x the hit rate on the sequential workload. The same effect (smaller magnitude)

shows up on the random workload because the pool is too small to keep the working set warm.

Experiment 2 -- Index Strategy Comparison

Setup: 500 records loaded, then 100 queries of one kind. Buffer pool = 16 frames, LRU. For the range row we use an integer primary key so the B+ tree fast path applies (B+ tree is on the PK; querying a non-PK int field would force a fallback).

Query Type	heap_scan	hash_index	bplus_tree
Equality	27378	14847	15979
Range	29343	19898 (fallback)	17009

Hash wins on equality (~46% reduction vs heap_scan): one bucket page plus one data page per lookup. B+ tree is close, paying a few extra index-node accesses on the way down. On range, B+ tree wins because it walks one root-to-leaf path then a short leaf-chain segment, touching only the data pages whose keys fall in [lo, hi]. Hash has no range support and falls back to heap_scan per spec 7.2; its number is slightly below pure heap_scan because no index bookkeeping runs.

Experiment 3 -- Buffer Pool Size Sensitivity

Setup: 500 records, 200 random equality queries. Strategy = bplus_tree, LRU. Buffer pool varies.

Buffer Size (frames)	Total I/Os	Hit Rate
4	19139	22.9%
8	17387	30.5%
16	16202	35.2%
32	11884	53.4%
64	1567	98.2%

Hit rate climbs monotonically with pool size, from 22.9% at 4 frames to 98.2% at 64. The biggest jump is between 32 and 64 frames -- that's where the working set (data pages + index nodes for our 500-record relation) finally fits, so almost every access becomes a hit. At 64 frames we do roughly 12x less disk work than at 4 frames. Below the working-set threshold, LRU thrashes.

7. Closing Notes

Validation summary: every layer ships with its own standalone test script. Together they verify round-trip page I/O, free-list reuse, persistence across reopen, LRU and MRU eviction order, sequential-flooding behaviour, dirty writeback correctness, pinned-page protection, slotted-page bitmap accounting, schema round-trip, catalog recovery, all five operations end-to-end under each of the three strategies, duplicate-PK detection, multi-level B+ tree construction (verified at 1000 keys, height 4), failure handling under garbage input, and log.csv persistence across runs.

Spec ambiguities we resolved explicitly. Identifier characters: spec 4.3 says type/field names are 'alphanumeric only', but the spec's own sample input uses `military_strength` (underscore). We accept underscore in identifiers; string field values stay strictly alphanumeric. search record extras: spec lists 'search record <type> <pk> <v1> ...' but the sample uses only 'search record house Atreides'. We accept both forms; the values after the PK are ignored on lookup. B+ tree fanout: a

natural 4096-byte page can hold hundreds of entries -- enough that the tree would remain a single node for any plausible workload. We capped per-node entries at `max_records_per_page` so the experiments can observe real B+ behaviour (a multi-level tree, range scans that follow leaf chains, etc.).

Files produced by the engine (next to `archive.py`): `output.txt`, `log.csv`, `stats_output.txt`, `<type>.dat`, `<type>.hidx` (for `hash_index` strategy), `<type>.bidx` (for `bplus_tree` strategy), `catalog.dat`.